Лабораторная работа №5: Практикум Midterm. Внедрение зависимостей (DI) и устранение нарушений LSP и DIP

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 5 из 10
Формат: Лабораторная работа (Midterm Practice), 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Комплексная проверка практических навыков проектирования объектно-ориентированных систем в рамках рубежного контроля (Midterm).
Студентам предстоит: 1. Выявить и исправить критическое нарушение **Liskov Substitution Principle (LSP)** в иерархии типов транспортных проездных документов. 2. Провести рефакторинг «толстого» интерфейса бортового терминала в соответствии с **Interface Segregation Principle (ISP)**. 3. Полностью инвертировать зависимости по **Dependency Inversion Principle (DIP)** и связать модули системы через контейнер внедрения зависимостей (**Dependency Injection Container**).

---

2. Структура репозитория и требования к сдаче

Работа выполняется в папке lab05_midterm_di/:

```
lab05_midterm_di/
├── contracts/
│   ├── __init__.py
│   ├── passes.py        # Исправленная иерархия проездных по LSP
│   └── hardware.py      # Ролевые протоколы оборудования по ISP
├── container/
│   ├── __init__.py
│   └── ioc.py           # Легковесный DI-контейнер
├── services/
│   ├── __init__.py
│   └── validator.py     # Валидатор с внедренными через DIP зависимостями
├── tests/
│   ├── __init__.py
│   ├── test_lsp.py      # Тесты подстановки Лисков
│   └── test_di.py       # Тесты резолвинга зависимостей и сквозной сценарий
└── README.md            # Отчет по рубежному контролю 1 (Midterm)
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Устранение нарушения LSP: разделение контрактов оплаты и верификации	5
Блок 2	Разделение толстого интерфейса на ролевые протоколы по ISP	5
Блок 3	Реализация DI-контейнера и связывание графа объектов без new внутри классов	6
Блок 4	Тестирование подстановки подтипов и изолированное тестирование компонентов	4

---

3. Практические задания

Блок 1. Исправление нарушения LSP (5 баллов)

Вам дан ошибочный базовый класс и его наследник:

```python
# ОШИБКА LSP
class BaseCard:
    def charge(self, amount: Decimal) -> None: ...

class SocialFreePass(BaseCard):
    def charge(self, amount: Decimal) -> None:
        raise Exception("Бесплатный проездной нельзя тарифицировать!")
```

Любой код, обрабатывающий коллекцию list[BaseCard], упадет при вызове charge().
Задание: 1. Разделите контракты на PassVerificationProtocol (проверка права на проезд) и BalanceChargeableProtocol (списание баланса). 2. Реализуйте классы: - StandardTransitCard: поддерживает и проверку, и списание. - SocialFreePass: поддерживает только проверку права на проезд (без списания баланса). 3. Докажите соблюдение LSP: любая функция, принимающая PassVerificationProtocol, работает одинаково корректно и со стандартной картой, и с социальным проездным без вызова исключений.

Блок 2. Разделение интерфейсов по ISP (5 баллов)

В файле contracts/hardware.py разбейте монолитный контракт валидатора на 3 независимых ролевых протокола: 1. CardScannerProtocol: методы scan_card() -> str и is_ready() -> bool. 2. VisualIndicatorProtocol: методы show_green(), show_red(reason: str). 3. AudioBuzzerProtocol: методы beep_success(), beep_error().
Спроектируйте класс валидатора OnboardValidator, который зависит только от этих трех узких протоколов.

Блок 3. Реализация DI-контейнера (6 баллов)

В файле container/ioc.py реализуйте класс Container:

```
class Container:
    def register_singleton(self, interface: type, implementation_or_instance: Any) -> None: ...
    def register_transient(self, interface: type, factory_func: Callable) -> None: ...
    def resolve(self, interface: type) -> Any: ...
```

В файле services/validator.py создайте функцию конфигурации приложения bootstrap_system(config: dict) -> Container: - Контейнер должен автоматически регистрировать репозитории, драйверы железа и тарифный сервис. - Создание экземпляра OnboardValidator должно происходить через container.resolve(OnboardValidator). Внутри классов запрещено использовать прямое инстанциирование зависимостей через ClassName().

Блок 4. Автоматическое тестирование (pytest) (4 балла)

В tests/test_lsp.py и tests/test_di.py: 1. Напишите тест, подтверждающий, что замена StandardTransitCard на SocialFreePass в функции турникета пропускает пассажира без исключений. 2. Напишите тест сборки графа зависимостей: при вызове container.resolve(OnboardValidator) возвращается полностью рабочий объект со всеми внедренными зависимостями. 3. Напишите тест подмены: зарегистрируйте в контейнере MockAudioBuzzer и проверьте, что реальный звук не воспроизводится, а счетчик вызовов мока равен 1.

```
pytest tests/ -v
```

---

4. Контрольные вопросы Midterm

1. Сформулируйте принцип подстановки Барбары Лисков через постусловия и предусловия.
2. В чем разница между временем жизни зависимостей Singleton и Transient в DI-контейнерах?
3. Почему Service Locator часто называют антипаттерном по сравнению с классическим Constructor Injection?
4. Как инверсия зависимостей (DIP) связана с понятием архитектурных границ (Architectural Boundaries)?